TDE2 — Busca Competitiva

Um agente alfa-beta para o Lig 4 e sua avaliação contra linhas de base

Guilherme Arcanjo de Morais Lucas da Costa Paula Renan da Silva Andrade
Renan Americo Herculano

PUCPR · Escola Politécnica · Bacharelado em Ciência da Computação
Inteligência Artificial (PCO118A) · Prof. Carlos Costa Cedro · RA2

Resumo

Construímos um agente de busca alfa-beta com profundidade fixa para o Lig 4 (Connect Four, 6×7) e o comparamos com um agente aleatório e com o próprio agente em profundidades menores. A corretude do alfa-beta foi verificada pela igualdade de valor com o Minimax em profundidade completa. Em 394 partidas, com custo medido em *nós visitados*, o AlfaBeta-d8 venceu as 100 partidas contra o aleatório, fez 0,811 ponto por partida contra o d2 e 0,694 contra o d4, mas apenas 0,520 contra o d6, sem diferença significativa. O custo por lance tem cauda pesada: mediana de alguns milhares de nós e máximo de 1.942.939. O agente não tem garantia de jogo ótimo nem limite de custo por lance, e o principal defeito medido é que a função de avaliação ignora a paridade das ameaças.

Sumário

1 O problema e por que ele é adequado ao método **2**

2 Formalização (ID2.1) **2**
2.1 Fator de ramificação, profundidade e tamanho do espaço 3
2.2 O que a representação torna impossível por construção 3

3 Implementação (ID2.2) **3**

4 Verificação de corretude **4**

5 Protocolo experimental (ID2.3) **5**

6 Resultados **6**
6.1 Resultado das partidas 6
6.2 Custo: nós visitados por lance 6
6.3 Ordenação de jogadas 7

7 Análise **8**

8 O que não funcionou e garantias ausentes **9**

9 Ferramenta de visualização **9**

10 Arquivos e reprodução **10**

1 O problema e por que ele é adequado ao método

Lig 4. Dois jogadores, X e O, alternam soltando uma peça em uma coluna não cheia de um tabuleiro vertical de 6 linhas × 7 colunas; a peça cai até a casa livre mais baixa. Vence quem alinhar 4 peças na horizontal, vertical ou diagonal. Tabuleiro cheio sem alinhamento é empate.

Tabela 1: As cinco verificações exigidas para o jogo.

Propriedade	Sim?	Justificativa
Dois jogadores	sim	X e O se alternam.
Soma zero	sim	A vitória de um é a derrota do outro e o empate vale 0: $\textsc{Utilidade}(s, O) = -\textsc{Utilidade}(s, X)$.
Informação perfeita	sim	O tabuleiro inteiro é visível para os dois jogadores.
Determinístico	sim	$\textsc{Resultado}(s, a)$ é único; não há sorteio.
Finito	sim	Cada jogada ocupa uma das 42 casas, então toda partida acaba em no máximo 42 jogadas.

O jogo é pequeno o bastante para implementar as regras em poucas linhas e grande demais para o Minimax puro (Seção 2.1). Por isso a poda alfa-beta, o corte de profundidade e a função de avaliação são necessários. Para a verificação usamos instâncias menores do próprio Lig 4 (Seção 4).

2 Formalização (ID2.1)

A implementação está em `lig4.py`. A classe `Lig4(linhas, colunas, k)` é genérica: as instâncias pequenas usadas na verificação (4×4 e 3×4 com $K = 3$) são o mesmo código com outros parâmetros.

Estado e s_0. $s = (x, o, \mathit{alturas}, \mathit{vez})$. Os campos x e o são *bitboards*: a casa (coluna c, linha r, com $r = 0$ embaixo) é o bit $c \cdot 7 + r$. Cada coluna ocupa 7 bits, 6 casas mais um bit sentinela sempre zero no topo, num total de 49 bits. $\mathit{alturas}[c] \in \{0, \ldots, 6\}$ é o número de peças na coluna c, e $\mathit{vez} \in \{X, O\}$ diz quem joga. Assim $\textsc{Jogador}(s) = s.\mathit{vez}$ é função do estado. s_0 é o tabuleiro vazio, com $\mathit{alturas} = (0, \ldots, 0)$ e $\mathit{vez} = X$: MAX começa.

Ações e Resultado. $\textsc{Ações}(s) = \{c \in \{0..6\} : \mathit{alturas}[c] < 6\}$, ou $\emptyset$ se s é terminal; logo $|\textsc{Ações}(s)| \leq 7$. $\textsc{Resultado}(s, c)$ liga o bit $c \cdot 7 + \mathit{alturas}[c]$ no bitboard de quem joga, incrementa $\mathit{alturas}[c]$ e passa a vez. Jogar em coluna cheia é erro (exceção).

Teste-Terminal. Verdadeiro se X ou O tem 4 em linha ou se as 42 casas estão ocupadas. O alinhamento é testado com deslocamentos: para cada direção $d \in \{1, 7, 6, 8\}$ (vertical, horizontal e as duas diagonais), há 4 em linha se $b \,\&\, (b \gg d) \,\&\, (b \gg 2d) \,\&\, (b \gg 3d) \neq 0$.

Utilidade$(s, \mathbf{MAX})$. Um único número, do ponto de vista de MAX:

$$\textsc{Utilidade}(s, \mathrm{MAX}) = \begin{cases} +(1000 - n) & \text{se X venceu,} \\ -(1000 - n) & \text{se O venceu,} \\ 0 & \text{empate,} \end{cases}$$

onde $n \leq 42$ é o número de peças no tabuleiro. O desconto por n depende só do estado e faz a vitória mais rápida valer mais, o que impede o agente de adiar indefinidamente uma vitória que já viu. Toda vitória vale pelo menos $1000 - 42 = 958$ em módulo, e a avaliação heurística é limitada a ± 900. Assim

$$|\textsc{Aval}(s)| \leq 900 < 958 \leq |\textsc{Utilidade}(s')| \quad \text{para todo corte } s \text{ e todo terminal com vencedor } s',$$

e um valor vindo de um corte nunca se confunde com o de uma vitória ou derrota provada.

2.1 Fator de ramificação, profundidade e tamanho do espaço

O fator de ramificação é 7 no início e cai quando colunas enchem. A média medida nos estados em que o agente jogou foi $b \approx 5{,}11$. A partida tem no máximo 42 jogadas; a duração média medida foi $m \approx 28{,}7$ (38,5 contra o d6). Daí

$$b^m \leq 7^{42} \approx 3{,}1 \times 10^{35} \text{ folhas}, \qquad 5{,}11^{\,28{,}7} \approx 2{,}2 \times 10^{20}.$$

O número de posições legais distintas é cerca de $4{,}5 \times 10^{12}$ (contagem de Tromp). Mesmo a 10^6 nós por segundo, percorrer 10^{20} folhas levaria mais de três milhões de anos, então o Minimax completo é inviável e usamos alfa-beta com corte de profundidade.

2.2 O que a representação torna impossível por construção

- **Peças flutuando.** Uma peça só entra na casa $(c, \mathit{alturas}[c])$, logo toda coluna é preenchida de baixo para cima, sem buracos.
- **Jogar em coluna cheia.** A coluna cheia não pertence a Ações(s) e Resultado a rejeita.
- **Jogar duas vezes seguidas** ou fora da vez. Resultado sempre inverte *vez*, e a peça vai para o bitboard de Jogador(s).
- **Casa ocupada pelos dois.** x e o nunca têm o mesmo bit ligado, porque só a casa livre mais baixa recebe peça.
- **Alinhamento "dando a volta" entre colunas.** O bit sentinela de cada coluna é sempre zero. Por isso o deslocamento horizontal ou diagonal que sairia do topo de uma coluna para a base da seguinte encontra um zero e o `&` se anula.

A representação não impede o estado "os dois venceram". Ele é inalcançável, porque o jogo para no primeiro alinhamento, e Ações devolve $\emptyset$ em qualquer estado com vencedor.

3 Implementação (ID2.2)

O código está em `busca.py`. O alfa-beta segue o pseudocódigo da aula. A unidade de custo é contada na primeira linha: *cada chamada da função recursiva é um nó visitado*, inclusive a raiz, as folhas cortadas por profundidade e os terminais.

Listing 1: `busca.py`: alfa-beta com contador de nós.

```
93  def alfabeta(jogo: Lig4, s: Estado, prof=None, aval=None, cont: Contador | None = None,
94               ordem=None, alfa=-math.inf, beta=math.inf):
95      """Valor minimax de s com poda alfa-beta (fail-hard dentro de [alfa, beta])."""
96      if cont is not None:
97          cont.nos += 1
98      if jogo.terminal(s):
99          return jogo.utilidade(s)
100     if prof == 0:
101         return aval(s)
102     p = None if prof is None else prof - 1
103     if jogo.jogador(s) == MAX:
104         v = -math.inf
105         for a in acoes_ordenadas(jogo, s, ordem):
106             v = max(v, alfabeta(jogo, jogo.resultado(s, a), p, aval, cont, ordem, alfa, beta))
107             if v >= beta:
108                 return v
109             alfa = max(alfa, v)
110         return v
111     v = math.inf
112     for a in acoes_ordenadas(jogo, s, ordem):
113         v = min(v, alfabeta(jogo, jogo.resultado(s, a), p, aval, cont, ordem, alfa, beta))
```

```
        if v <= alfa:
            return v
        beta = min(beta, v)
    return v
```

Na raiz, `decidir` examina todas as colunas. Cada filho é buscado com a janela que só aceita valores estritamente melhores que o melhor já encontrado. A jogada devolvida é a primeira de maior valor na ordem de exame, então o agente é determinístico.

Ordenação de jogadas. Centro primeiro, na ordem fixa 3, 2, 4, 1, 5, 0, 6, com custo zero. A coluna central participa de 60 das 276 inclusões casa-janela e a de borda, de 24 (Figura 1). Por isso as jogadas centrais costumam ser as melhores e provocam podas cedo. O efeito medido está na Seção 6.3.

Função de avaliação. Uma *janela* é um segmento de 4 casas em linha; o tabuleiro tem 69. Uma janela com peças dos dois jogadores vale 0, porque ninguém mais vence por ela. Para as demais:

$$\begin{aligned}\textsc{Aval}(s) = &\sum_{\text{janelas}} \left(5 \cdot [\text{3 de X + 1 vazia}] + 2 \cdot [\text{2 de X + 2 vazias}]\right) \\ &- \sum_{\text{janelas}} \left(5 \cdot [\text{3 de O + 1 vazia}] + 2 \cdot [\text{2 de O + 2 vazias}]\right) \\ &+ 3 \cdot (\text{peças de X} - \text{peças de O na coluna central}),\end{aligned}$$

limitada a ± 900. Os pesos estão na Tabela 2. Eles foram escolhidos à mão, sem ajuste, e são os mesmos para todos os agentes de busca.



Figura 1: Janelas de 4 casas que passam por cada casa.

Tabela 2: Características e pesos da Aval.

Característica	Peso	Justificativa
3 em linha + 1 vazia	5	ameaça direta, a uma jogada da vitória
2 em linha + 2 vazias	2	ameaça em construção, faltam duas jogadas
peça na coluna central	3	a casa central está em 13 janelas, um canto em 3: controle posicional

Mecanismo de parada. Profundidade fixa d, cortando também nos terminais: $d = 8$ para o agente avaliado e $d \in \{2, 4, 6\}$ para os adversários. Preferimos profundidade fixa a aprofundamento iterativo com relógio porque o custo é medido em nós. Com relógio, o número de nós e as próprias jogadas dependeriam da máquina, e o experimento deixaria de ser reprodutível. Não há busca de quiescência (Seção 8).

4 Verificação de corretude

Pelo Teorema 1 da Aula 7, aplicado à mesma árvore, o alfa-beta devolve na raiz exatamente o valor do Minimax. A igualdade de valor em profundidade completa é, portanto, a evidência de corretude que se pode exigir. Ela distingue um alfa-beta correto de um que poda demais. O script `verificacao.py` compara os dois nos casos da Tabela 3 e terminou com **APROVADO**.

Tabela 3: Minimax × alfa-beta. Nós visitados somados sobre todas as posições de cada caso.

Caso	Minimax	Alfa-beta	Divergências
4×4, $K = 3$, profundidade completa a partir de s_0, sem ordenação	valor 991, 6.761.217	valor 991, 33.810	0
4×4, $K = 3$, profundidade completa a partir de s_0, centro primeiro	valor 991, 6.761.217	valor 991, 11.985	0
200 posições aleatórias do 3×4, $K = 3$, profundidade completa	7.627.105	114.733 (1,5 %)	0
100 finais aleatórios do 6×7 com 8 casas vazias, profundidade completa	61.291	7.614 (12,4 %)	0
100 posições do 6×7 com corte em $d = 4$ e a mesma AVAL (valor e jogada)	238.275	26.355 (11,1 %)	0

O valor $991 = 1000 - 9$ significa que, no 4×4 com $K = 3$, X vence com jogo perfeito e a vitória sai na 9.ª peça. O mesmo valor é obtido com e sem ordenação, enquanto o número de nós cai de 6.761.217 para 11.985. O Teste 5 cobre o visualizador (Seção 9). As versões instrumentadas da busca (`rastreio.py`), que registram a árvore para a animação, foram comparadas com as de `busca.py` em 240 casos: mesmo valor, mesma jogada e o mesmo número de nós, sem divergência.

5 Protocolo experimental (ID2.3)

Agentes. O agente avaliado é o **AlfaBeta-d8**: alfa-beta, profundidade 8, AVAL de janelas e centro primeiro. Os adversários são:

1. **Aleatório**: escolha uniforme em AÇÕES(s) com semente fixa. É a linha de base trivial.
2. **AlfaBeta-d2, d4 e d6**: o próprio agente com profundidade menor, com o mesmo código, a mesma AVAL, os mesmos pesos e a mesma ordenação.

Unidade de custo. Nós visitados, definidos na Seção 3. Para cada lance do agente avaliado registramos o número de nós e reportamos média, desvio, mínimo, quartis, mediana, p95 e máximo. O tempo de parede é gravado só como informação e não entra em nenhuma comparação.

Partidas e sementes. Dois agentes determinísticos jogariam sempre a mesma partida. Por isso:

- contra AlfaBeta-dk, as 2 primeiras jogadas são forçadas e percorrem as $7 \times 7 = 49$ aberturas. Cada abertura é jogada 2 vezes, trocando quem começa, num total de 98 partidas por confronto. Não há sorteio. Empates entre jogadas são desfeitos pela ordem de exame, e as jogadas forçadas não contam custo;
- contra o Aleatório, 100 partidas, com a semente do adversário igual ao número da partida (0 a 99). O agente avaliado começa nas partidas pares.

São 394 partidas no total. O teste de ordenação sorteia, com semente 42, 300 posições em que o agente avaliado jogou e as busca com d8 com e sem ordenação.

Calibração simétrica. Os quatro agentes de busca usam o mesmo código, a mesma AVAL e a mesma ordenação; só a profundidade muda. Nenhum dos lados foi ajustado: os pesos (5, 2, 3) foram fixados antes do experimento e não foram varridos.

Reprodução. Todo o experimento é determinístico:

```
python3 verificacao.py  ·  python3 experimento.py  ·  python3 exportar_dados.py
```

O `experimento.py` grava em `resultados.json`, para cada partida, a sequência de colunas, o resultado e os nós de cada lance. Rodar de novo produz os mesmos números (só o tempo de parede muda), e o visualizador reproduz qualquer partida e confere os dados lance a lance (Seção 9).

6 Resultados

6.1 Resultado das partidas

Tabela 4: Resultados do AlfaBeta-d8. IC 95 % de Wilson para a taxa de vitória; pontos: vitória = 1, empate = ½.

Adversário	Partidas	V	E	D	Taxa de vitória (IC 95 %)	Pontos
Aleatório	100	100	0	0	100,0 % (96,3 % a 100,0 %)	1,000
AlfaBeta-d2	98	77	5	16	78,6 % (69,5 % a 85,5 %)	0,811
AlfaBeta-d4	98	61	14	23	62,2 % (52,4 % a 71,2 %)	0,694
AlfaBeta-d6	98	38	26	34	38,8 % (29,7 % a 48,7 %)	0,520

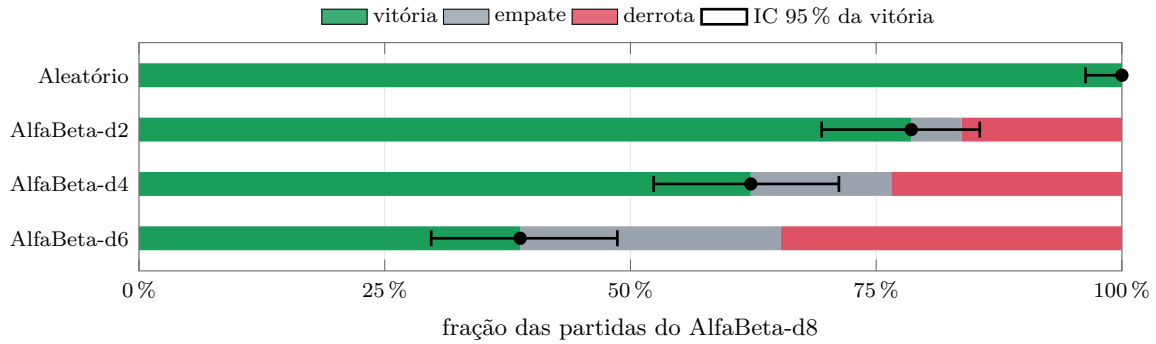


Figura 2: Vitórias, empates e derrotas do AlfaBeta-d8 por adversário, com o IC 95 % da taxa de vitória.

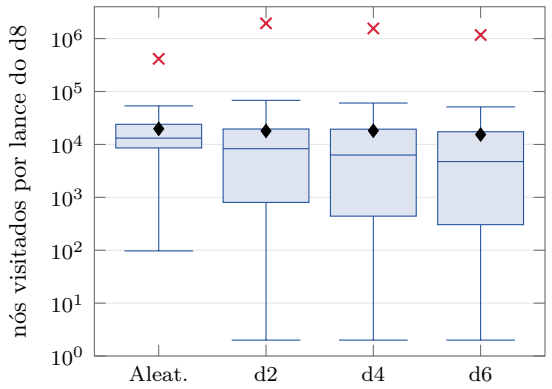
Tabela 5: Resultado por lado (V/E/D do AlfaBeta-d8).

Adversário	d8 começando (X)	d8 em segundo (O)
Aleatório	50/0/0	50/0/0
AlfaBeta-d2	43/4/2	34/1/14
AlfaBeta-d4	25/9/15	36/5/8
AlfaBeta-d6	15/15/19	23/11/15

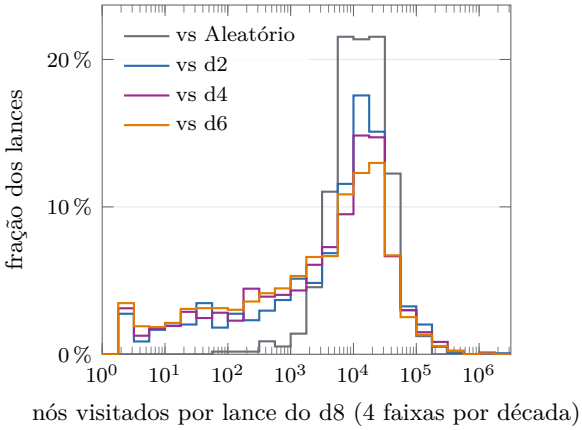
6.2 Custo: nós visitados por lance

Tabela 6: Nós visitados por lance do AlfaBeta-d8. A última coluna é a média do adversário, na mesma unidade.

Adversário	Lances	Média	Desvio	Q_1	Mediana	Q_3	p95	Máx.	Adv. (média)
Aleatório	571	19.727	25.383	8.580	13.149	24.005	53.476	414.620	0
AlfaBeta-d2	1.384	17.980	60.145	801	8.296	19.555	68.122	1.942.939	20
AlfaBeta-d4	1.664	18.132	57.694	440	6.296	19.414	60.550	1.554.522	196
AlfaBeta-d6	1.788	15.332	42.276	304	4.732	17.346	51.179	1.170.289	1.643



(a) Caixa: quartis; bigodes: mínimo e p95; ×: máximo; ♦: média.



(b) Distribuição completa, com 4 faixas por década.

Figura 3: Distribuição dos nós visitados por lance do AlfaBeta-d8 (escala logarítmica).

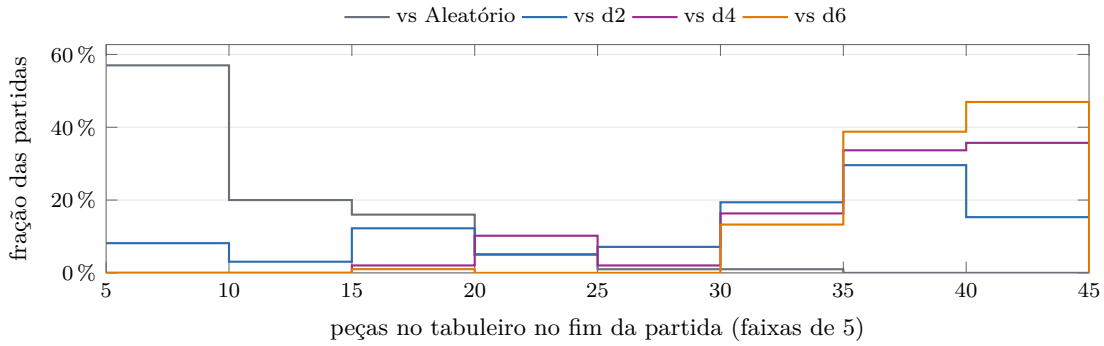
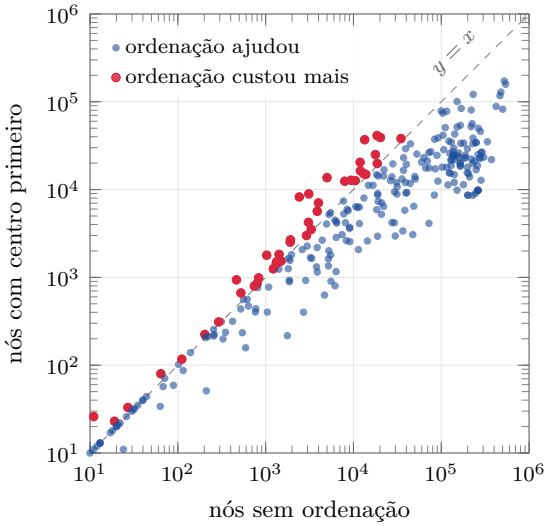


Figura 4: Duração das partidas (peças no fim). Média: 10,9 contra o aleatório, 29,9 contra o d2, 35,8 contra o d4 e 38,5 contra o d6.

6.3 Ordenação de jogadas



Nós (d8)	com	sem
Média	15.031	71.884
Mediana	6.754	13.623
p95	53.969	272.818
Máximo	172.133	540.669

Razão com/sem: média 0,60, mediana 0,47, máxima 3,41. Valor da raiz idêntico nas duas variantes: sim. Em 41 das 300 posições (pontos vermelhos, acima da diagonal) a ordenação custou *mais* nós.

Figura 5: Centro primeiro × ordem natural nas mesmas 300 posições, buscadas com d8.

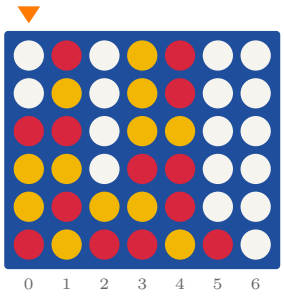
7 Análise

A linha de base é vencida sem exceção. Foram 100 vitórias em 100 contra o aleatório (IC 95 % 96,3 % a 100,0 %), com partidas curtas (mediana de 9 peças). Esse resultado só mostra que o agente não é trivial.

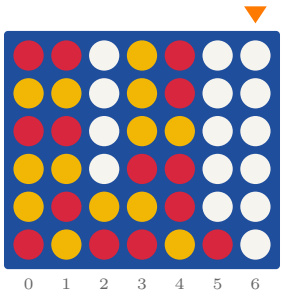
Os retornos da profundidade caem rápido. Os pontos por partida caem de 0,811 (contra o d2) para 0,694 (d4) e 0,520 (d6). Contra o d6, o IC 95 % da taxa de vitória (29,7 % a 48,7 %) fica inteiro abaixo de 50 %, e a pontuação está perto de ½ por causa dos 26 empates. Com 98 partidas não é possível afirmar que d8 > d6. Nossa hipótese é que, a partir daí, a Aval limita mais que a profundidade; o caso de zugzwang abaixo é um exemplo.

O custo por lance tem cauda pesada. A mediana fica entre 4.732 e 13.149 nós, mas o máximo chega a 1.942.939 (Figura 3). A profundidade fixa não limita o pior caso, por isso reportamos também p95 e máximo. O custo mediano cai contra adversários mais fortes, provavelmente porque as partidas ficam mais longas (Figura 4), e nas posições com colunas cheias há menos jogadas legais. O adversário gasta uma fração disso: em média 20, 196 e 1.643 nós por lance para d2, d4 e d6.

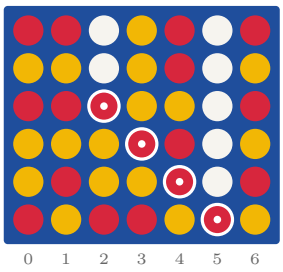
A Aval ignora a paridade das ameaças (zugzwang). O d8 perdeu 16 partidas até para o d2, 14 delas jogando em segundo, todas terminadas com 35 a 39 peças. A Figura 6 acompanha a partida nº 3 (abertura 0, 1). Com 25 peças no tabuleiro, a busca d8 ainda dava −5, ou seja, uma posição levemente favorável a O. Duas jogadas depois, com 27 peças, já via a vitória forçada de X (+965, isto é, X alinha com a 35.ª peça), tarde demais. No fim do Lig 4, quem é obrigado a jogar embaixo da ameaça do outro perde. Ameaças em linha ímpar ou par decidem esses finais, e a Aval conta as duas da mesma forma.



(a) 25 peças, valor −5; O joga na coluna 0



(b) 27 peças, valor +965: derrota já forçada



(c) final: X alinha na diagonal com a 35.ª peça

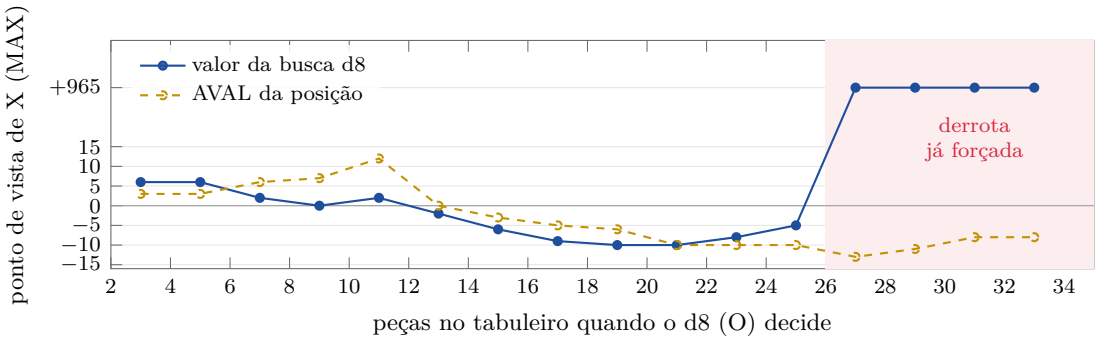


Figura 6: Partida nº 3 contra o AlfaBeta-d2, d8 jogando com O (amarelo). Em cima, posições em que o d8 decide (a seta marca a coluna escolhida). Embaixo, a Aval e o valor da busca d8 a cada decisão, do ponto de vista de X. O valor +965 é a vitória forçada de X.

A ordenação ajuda na média, mas nem sempre. Centro primeiro reduz a média de nós em 79 % e devolve o mesmo valor. Ainda assim, em 41 das 300 posições custou mais, até 3,41 vezes (Figura 5). Uma ordem fixa não sabe qual coluna é boa naquela posição.

8 O que não funcionou e garantias ausentes

- **O agente não é ótimo.** No 6×7 a busca não chega aos terminais, e o valor de um corte é uma estimativa. Só os valores $|v| \geq 958$ são vitórias provadas.
- **Efeito horizonte.** Não há busca de quiescência, então uma ameaça logo depois da profundidade 8 passa despercebida. A partida da Figura 6 é um caso medido.
- **Paridade das ameaças ignorada.** É o defeito da AVAL que levou à derrota da Figura 6. Achamos que ele explica boa parte das derrotas para o d2 jogando em segundo, mas só essa partida foi analisada em detalhe.
- **Pesos não ajustados.** Os valores (5, 2, 3) foram escolhidos à mão. Não sabemos se outros pesos seriam melhores.
- **Sem limite de custo por lance.** Com profundidade fixa, o pior caso medido foi 1.942.939 nós; nada impede um lance mais caro em outra posição.
- **Validade restrita às aberturas testadas.** Contra os adversários alfa-beta, os resultados valem para as 49 aberturas de 2 jogadas, não para qualquer partida. Como os dois lados são determinísticos, cada abertura gera uma única partida.
- **Determinismo explorável.** O mesmo estado sempre produz a mesma jogada, e um adversário que conheça o agente pode repetir uma linha vencedora.
- **Sem significância contra o d6.** A diferença d8 × d6 não é distinguível com 98 partidas.

9 Ferramenta de visualização

O `visualizador.py` é uma aplicação Tkinter, só com a biblioteca padrão, que roda o mesmo código de `lig4.py` e `busca.py`. As versões instrumentadas de `rastreio.py` só registram o que a busca faz; o Teste 5 de `verificacao.py` confere que elas dão o mesmo valor, a mesma jogada e o mesmo número de nós. A busca roda numa thread separada, então o contador de nós sobe na tela enquanto o agente pensa.

Jogar
: Tabuleiro com queda animada das peças e qualquer combinação de humano, aleatório ou AlfaBeta-d2 a d8 nos dois lados, com abertura forçada opcional. O painel mostra *nós visitados neste lance* ao vivo, totais por jogador e um gráfico de nós por lance em escala log. Acima de cada coluna aparece o valor que a busca atribuiu àquela jogada: exato, ou limite $\leq/\geq$ quando a janela da raiz cortou, e "vence em k" para vitórias forçadas. Há ainda a decomposição da AVAL da posição atual e o mapa de janelas por casa da Figura 1. A opção *Reproduzir partida do experimento* refaz qualquer das 394 partidas e confere resultado, peças, sequência de colunas e nós de cada lance contra o `resultados.json`.

Árvore alfa-beta
: Desenha a árvore completa do Minimax para uma posição pequena (finais em profundidade completa ou meio-jogo com $d \leq 3$) e anima o alfa-beta passo a passo, com α e β em cada nó do caminho, valores subindo e subárvores podadas em cinza. Os contadores comparam os nós do alfa-beta com os do Minimax, e o valor das duas buscas aparece lado a lado.

Resultados
: Os gráficos da Seção 6 lidos do `resultados.json`.

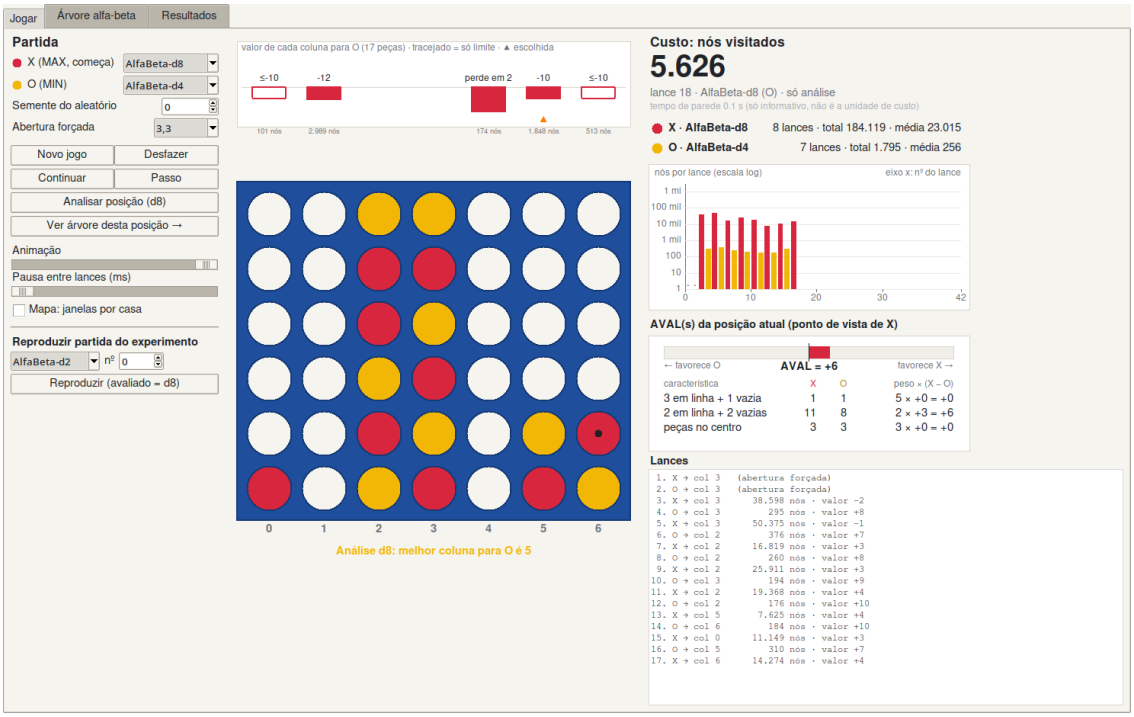
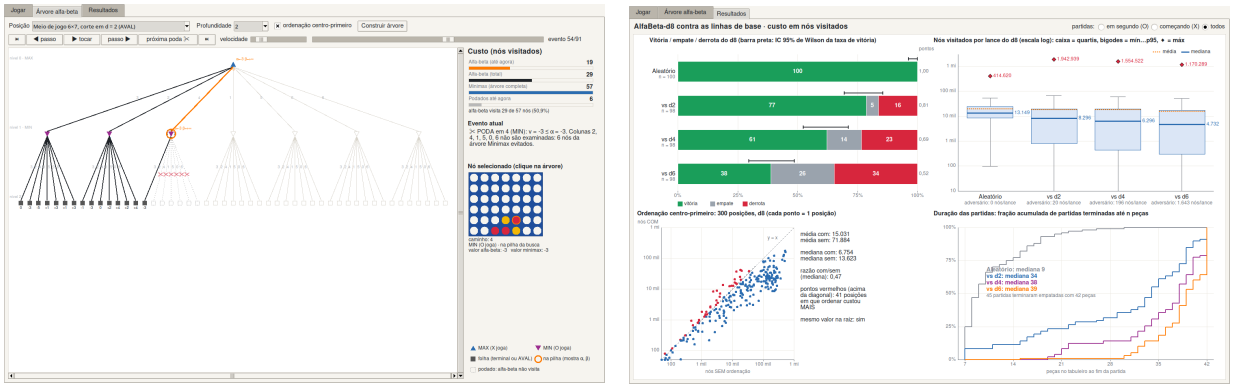


Figura 7: Aba *Jogar*: contador de nós ao vivo, valor de cada coluna na raiz e decomposição da AVAL.



(a) Árvore alfa-beta. **(b)** Resultados.

Figura 8: Demais abas do visualizador.

10 Arquivos e reprodução

Arquivo	Conteúdo
`lig4.py`	formalização: s_0, JOGADOR, AÇÕES, RESULTADO, TESTE-TERMINAL, UTILIDADE
`busca.py`	Minimax, alfa-beta, AVAL, ordenação, contador de nós, agentes
`verificacao.py`	igualdade Minimax = alfa-beta (saída em `verificacao_saida.txt`)
`experimento.py`	protocolo completo; gera `resultados.json` e `resultados.md`
`exportar_dados.py`	tabelas, números e tabuleiros deste relatório (`relatorio/dados/`)
`visualizador.py`, `viz/`, `rastreio.py`	ferramenta de visualização
`relatorio/`	este relatório, os slides e a ficha (`make` gera os três PDFs)